gems/activerecord-2.3.2/lib/active_record/attribute_methods.rb

```
# Generates all the attribute related methods for columns in the database
# accessors, mutators and query methods.
def define_attribute_methods
  return if generated_methods?
  columns_hash.each do |name, column|
    unless instance_method_already_implemented?(name)
      if self.serialized_attributes[name]
        define_read_method_for_serialized_attribute(name)
      elsif create_time_zone_conversion_attribute?(name, column)
        define_read_method_for_time_zone_conversion(name)
      else
        define_read_method(name.to_sym, name, column)
      end
    end

    unless instance_method_already_implemented?("#{name}=")
      if create_time_zone_conversion_attribute?(name, column)
        define_write_method_for_time_zone_conversion(name)
      else
        define_write_method(name.to_sym)
      end
    end

    unless instance_method_already_implemented?("#{name}?")
      define_question_method(name)
    end
  end
end
```

The instance_method_already_implemented? method is there to prevent involuntary *Monkeypatches (16)*: if a method by the name of the attribute already exists, then this code skips to the next attribute. Apart from that, the previous code does little but delegate to other methods that do the real work, such as define_read_method or define_write_method.

As an example, take a look at define_write_method. I've marked the most important lines with arrows:

gems/activerecord-2.3.2/lib/active_record/attribute_methods.rb

```
➤ def define_write_method(attr_name)
➤   evaluate_attribute_method attr_name,
➤     "def #{attr_name}=(new_value);write_attribute('#{attr_name}', new_value);end",
➤     "#{attr_name}="
➤ end

➤ def evaluate_attribute_method(attr_name, method_definition, method_name=attr_name)
    unless method_name.to_s == primary_key.to_s
      generated_methods << method_name
    end
```